A087

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

Name : Devyani Kamat SAP ID : 60017240102

EXPERIMENT NO. 3

AIM:

To construct layout compositions using Next.js root layouts and localized sub-layouts to safely maintain state during client-side transitions.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Nested Layout Architecture (App Router):** Next.js App Router organizes user interfaces through dynamic segment nesting. A layout.tsx file wraps child pages (children) and persists across route navigations within its directory scope. Unlike standard page components, layouts do not unmount during navigation between sibling routes in the same subtree.
- **Partial Rendering & State Preservation:** During client-side routing via Next.js <Link> components, Next.js performs partial rendering. It evaluates and re-renders only the dynamic leaf segments (page.tsx) while keeping parent layouts mounted in React's component tree. Consequently, local React state (useState, useRef), active timers (useEffect), and media playback initialized within a localized sub-layout remain entirely uninterrupted across route transitions.
- **Layouts (layout.tsx) vs. Templates (template.tsx):** While layouts preserve state and maintain their existing DOM nodes during navigation, templates create a new component instance on every navigation pass. Replacing a layout.tsx with a template.tsx forces React to reset internal hook states and re-mount DOM elements on each route change.

STEPS / ALGORITHM:

Step 1: Create the Root Layout (app/layout.tsx)

Define the global document structure and top-level navigation container.

```
      <h3>Post Page: <span style={{ color: '#15960b'
}}>{slug}</span></h3>
      <p>The page content rendered dynamically, while the
outer timer and counter in the sub-layout remained
unchanged.</p>
    </div>
  );
}
```

Step 6: Run and Validate Layout State Retention

Execute the development server:

```
npm run dev
```

1. Navigate to http://localhost:3000/blog.
2. Click **"Increment State Counter"** to modify the internal state.
3. Switch between **Post 1**, **Post 2**, and **Blog Index** using client links. Observe that the timer continues incrementing and the counter holds its value without resetting.

OBSERVATION:

Observe how navigating between dynamic routes updates the inner viewport without triggering an unmount cycle on BlogLayout. The Uptime Timer interval continues ticking continuously, and the State Counter value is strictly preserved.

Note that navigating back to the root route (/) unmounts the localized sub-layout segment. Re-entering /blog instantiates a new sub-layout component instance, resetting the state variables back to their initial values (0).

OUTPUT:

Development Terminal Execution:

```
▲ Next.js 16.x
  - Local:        http://localhost:3000
  - Environments: .env

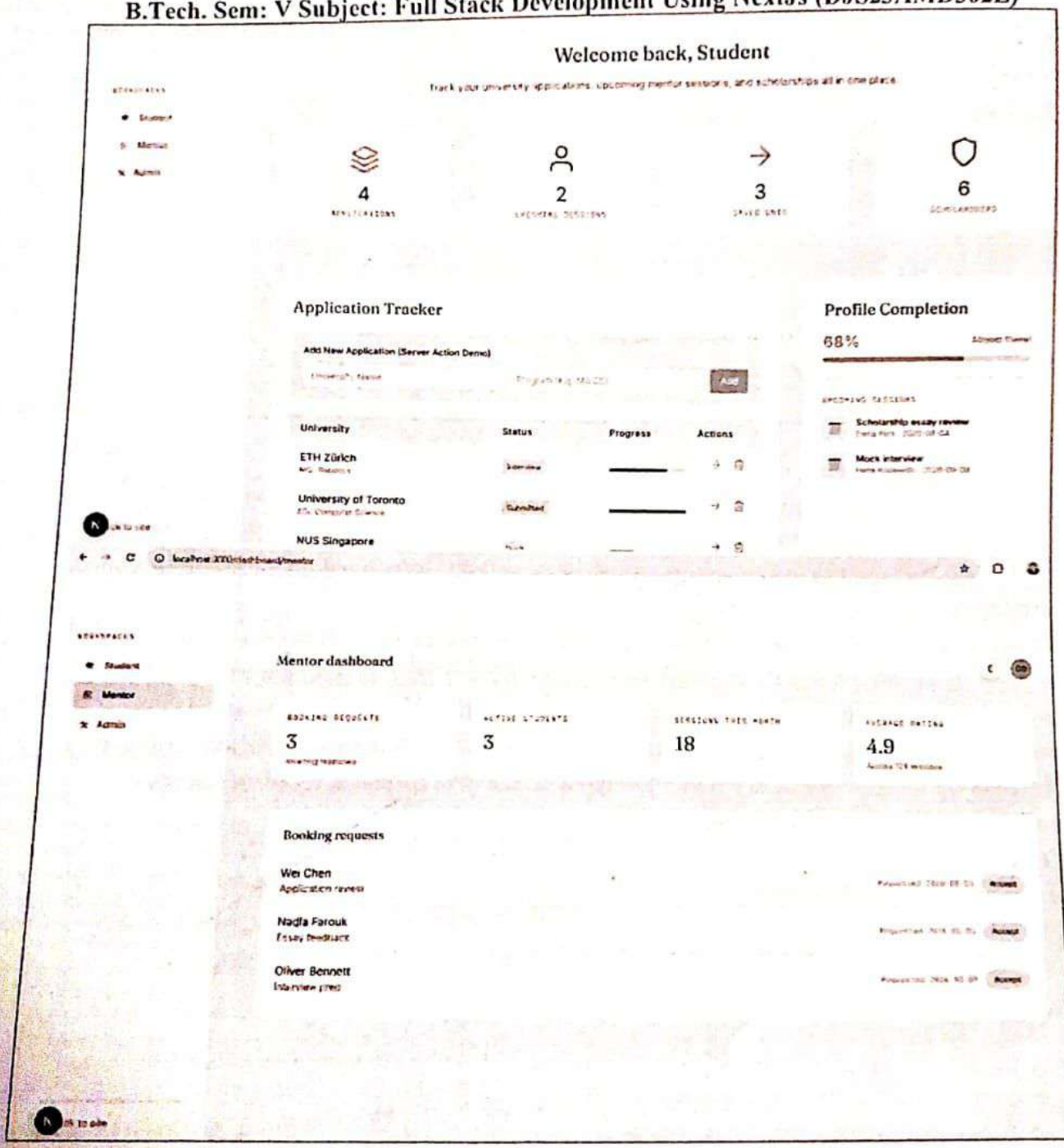
 ✓ Starting...
 ✓ Ready in 980ms
 ○ Compiling /blog/[slug] ...
 ✓ Compiled /blog/[slug] in 240ms
GET /blog/post-1 200 in 45ms
```

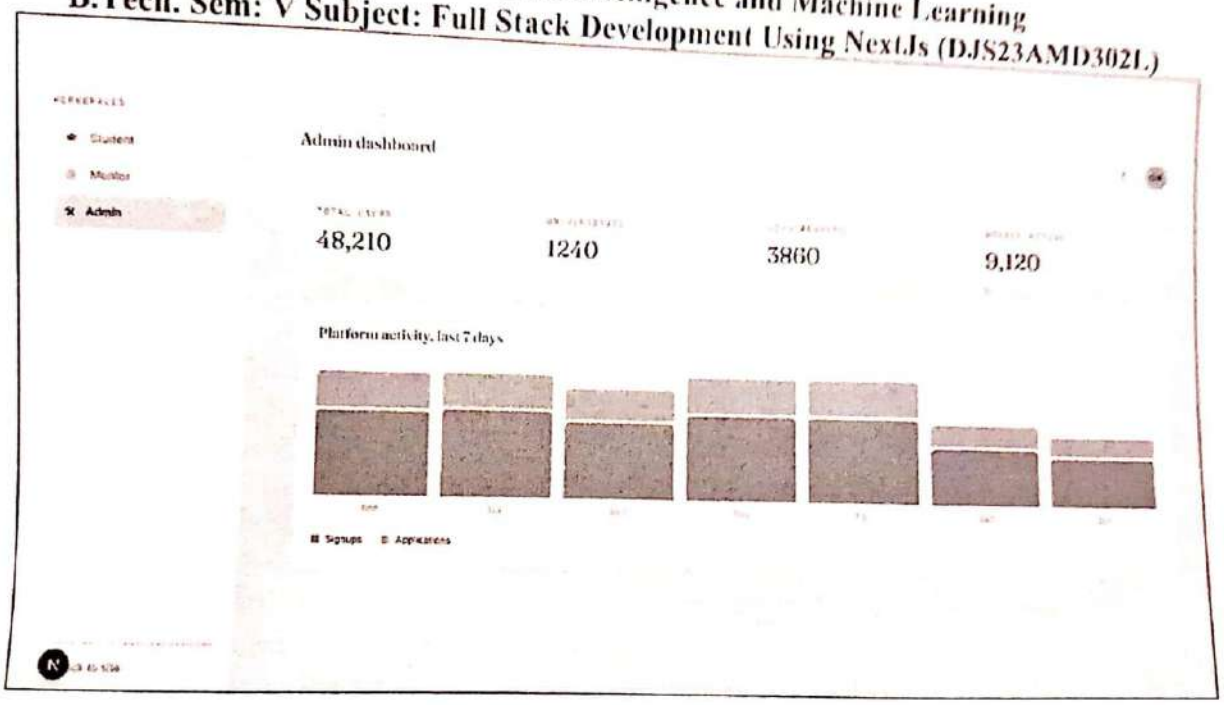
Observation :

Construction of layout compositions using Next.js root layouts was done.

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)





Rendered Viewport Verification:

- ***Sibling Sub-Route Navigation :*** Layout remains mounted. Uptime timer ticks continuously (12s, 13s, 14s...), and counter remains unchanged (5). Only the dynamic dynamic page slug header updates.
- **Root Scope Navigation:** The subtree unmounts completely. Navigating back to /blog resets Uptime Timer and State Counter to 0.

Conclusion :

Next.js root layout & localised sub-layers were used.